

RL vs PID for Adaptive Cruise Control

with different driving styles

Ruby Zhou

1 Introduction

Since I got rear ended yesterday, I thought it would be fun to do a report on adaptive cruise control. Adaptive cruise control is a driver-assistance system where a vehicle automatically adjusts its acceleration and braking to maintain a safe distance behind another vehicle. Basically, it follows the car in front of it in a safe and efficient way. However, unlike regular cruise control, which only tries to maintain a target speed, adaptive cruise control responds to changes in the speed of the lead vehicle, such as braking and acceleration. In this project, I simulate a simplified adaptive cruise control problem with two cars moving in one dimension. The **lead car** follows a scripted speed pattern, while the **ego car** is controlled by either a PID controller or a reinforcement learning policy. The ego car's goal is to follow the lead car safely and smoothly while maintaining a desired following distance. The main question I'm asking is:

Can a reinforcement learning policy learn adaptive cruise control behaviour comparable to a tuned PID controller?

I also looked at a few research papers to see how other people approached this problem. Lin et al. compared deep reinforcement learning with model predictive control for adaptive cruise control, and they also treated acceleration as a continuous action [1]. SAINT-ACC treats adaptive cruise control as a balance between safety, comfort, and traffic efficiency instead of only trying to minimize distance error [2]. I also found research on personalized adaptive cruise control, where different cost functions are used to represent different driver preferences [3]. This was useful for the driving style part of my project.

The main things I ended up investigating were:

- how much tuning changes the PID baseline;
- how the reward function changes what PPO actually learns;
- why some random seeds converge while other seeds completely fail;
- whether the policy generalizes to lead-car behaviour it did not train on; and
- how the same ACC setup can produce aggressive, safe, or smooth behaviour.

2 Related Work

Classical vs. Learning-Based ACC

Traditional adaptive cruise control normally uses feedback control or model predictive control. The nice thing about these methods is that they are easier to understand and much cheaper to run than a neural network. Deep reinforcement learning is different because it learns a policy from experience instead of being directly told a control equation.

Lin et al. used DDPG for adaptive cruise control and compared it against MPC [1]. Their setup was especially relevant to mine because the action was also continuous acceleration, and the controller had to balance tracking, comfort, and control effort. SAINT-ACC also treats ACC as a multi-objective problem instead of only caring about distance error [2].

Why Multiple Seeds Matter

A big issue with deep RL is that the exact same setup can give different results depending on the random seed. The neural network starts with different random weights, the environment samples different experiences, and the training path can end up somewhere completely different. Henderson et al. explain why reporting only one lucky run can make an RL method look much more reliable than it actually is [5]. This is why I started using five-seed sweeps instead of only training one model.

3 Simulator Setup

The simulator contains two vehicles: the lead car and the ego car. Both cars move along a straight one-dimensional road. Each car has a position, speed, and acceleration.

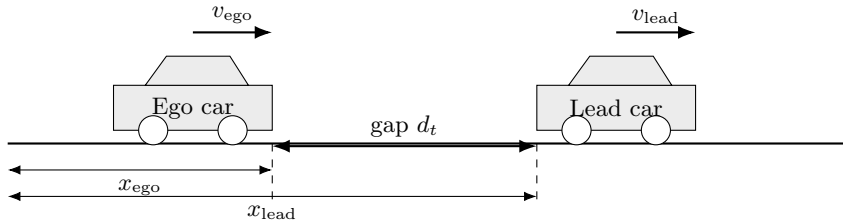


Figure 1: Schema of the 1D ACC simulator.

The basic vehicle dynamics are:

$$v_{t+1} = v_t + a_t \Delta t \quad (1)$$

$$x_{t+1} = x_t + v_{t+1} \Delta t \quad (2)$$

where x is position, v is velocity, a is acceleration, and $\Delta t = 0.05$ s for PID and $\Delta t = 0.1$ s for RL. Both cars are initialised with $v_0 = 20$ m/s, and the lead car starts 30 m ahead of the ego car. Speeds are clipped to $[0, 35]$ m/s so cars cannot reverse.

The ego car is the controlled vehicle. Its action is acceleration:

$$a_{\text{ego}} \in [-5, 3] \text{ m/s}^2$$

This value is clipped to represent realistic braking and acceleration limits.

Scenarios

I scripted an acceleration profile for the lead car to follow for a 30 second episode. Two fixed scenarios are defined below:

Training, Validation, and Evaluation Split

At first I only had 2 scenarios, but I quickly realized that the RL policies would overfit and memorize the fixed scenarios (timing of lead-car movements, like “brake at $t = 5$ s”, instead of learning to react to distance error and relative speed.) The PID controller doesn’t have this problem because it only responds to instantaneous measurements, not to time.

To ensure a fair comparison, scenarios are assigned to three roles:

- **RL Training:** A randomized scenario where braking, accelerating, and cruising are sampled from random distributions at the start of each episode. This forces the RL policy to learn a reactive control strategy instead of memorizing.

Default (easy):

- 0–5s: cruise at constant speed (0 m/s^2)
- 5–10s: slow down (-1 m/s^2)
- 10–15s: speed up (1 m/s^2)
- 15–22s: cruise again (0 m/s^2)
- 22–30s: moderate brake (-2 m/s^2)

Hard (aggressive):

- 0–3s: cruise (0 m/s^2)
- 3–6s: emergency brake (-4 m/s^2)
- 6–10s: stopped or near-stopped (0 m/s^2)
- 10–13s: hard acceleration (2 m/s^2)
- 13–18s: cruise (0 m/s^2)
- 18–21s: hard brake (-3 m/s^2)
- 21–25s: moderate recovery (1.5 m/s^2)
- 25–30s: gentle slowdown (-1 m/s^2)

Figure 2: Default and hard lead-car acceleration scenarios.

- **PID Tuning:** The PID gains are optimized with Bayesian search using Optuna across 20 fixed random profiles generated with different seeds.
- **Evaluation:** Both controllers are evaluated on the **hard** scenario for the first time.

4 PID Controller

The PID controller is used as the classical control baseline. Its goal is to keep the ego car at a desired distance behind the lead car.

The following distance is defined as:

$$d = x_{\text{lead}} - x_{\text{ego}}$$

The distance error is:

$$e(t) = d(t) - d_{\text{desired}}$$

where d_{desired} is the desired following distance. In my simulator, I use:

$$d_{\text{desired}} = 20 \text{ m}$$

If $e(t)$ is positive, the ego car is too far behind the lead car and should accelerate. If $e(t)$ is negative, the ego car is too close and should brake.

The PID controller computes acceleration using:

$$a(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

where:

- K_p , the **proportional term**, reacts to the current gap error: the actual distance vs desired bumper-to-bumper distance.
- K_i , the **integral term**, reacts to accumulated error over time. If the car stays slightly too close for a long time, then the integral term builds up and corrects the persistent error.
- K_d , the **derivative term**, reacts to how quickly the error is changing. If the ego car is getting closer to the lead car very quickly, the derivative term helps the ego car brake earlier instead of reacting too late.

5 PID Tuning

To find a strong PID baseline, I tuned the controller by evaluating each set of gains using a score that combined tracking accuracy, safety, comfort, and control effort.

The metrics used for tuning were:

$$\text{MAE} = \frac{1}{T} \sum_{t=1}^T |d_t - d_{\text{desired}}| \quad (3)$$

$$\text{MeanAbsJerk} = \frac{1}{T-1} \sum_{t=2}^T \left| \frac{a_t - a_{t-1}}{\Delta t} \right| \quad (4)$$

$$\text{ControlEffort} = \sum_{t=1}^T |a_t| \quad (5)$$

I did not want to judge the controllers using only one number. A controller could get very low distance error by following too closely, or get very low jerk just because it barely reacts. That is why I kept several metrics.

- **Mean Absolute Distance Error:** how close the ego car stayed to the desired distance (lower is better)
- **Minimum Following Distance:** if the minimum distance is too small, the ego car got dangerously close to the lead car (measures safety).
- **Mean Absolute Jerk:** acceleration change over time, high jerk means uncomfortable or sudden driving, like sudden braking or acceleration.
- **Control Effort:** how much the controller used acceleration or braking, so a lower effort would mean smoother + more efficient driving.
- **Collision Count:** counts the number of collisions (should always be 0!)

Grid Search

I first used a brute-force grid search over $K_p \in [0.20, 0.50]$, $K_i \in [0.0, 0.005]$, and $K_d \in [0.8, 2.0]$, evaluating all 210 combinations. The best result was:

$$K_p = 0.5, \quad K_i = 0.002, \quad K_d = 2.0 \quad (\text{score} = 5.69)$$

Grid search only checks the exact values I choose, so it can easily miss better values between outside the range. I also noticed that the best gains were always at the edge of the search range, which suggested that the true best values were probably outside the grid. After manually changing gains and re-running plots a few times, I realized I was mostly guessing, so I switched to Bayesian optimization with Optuna.

Bayesian Optimization

To search more effectively, I switched to Bayesian optimization using the TPE (Tree-structured Parzen Estimator) sampler from the Optuna library. Unlike grid search, Bayesian optimization intelligently focuses trials on promising regions of the parameter space rather than exhaustively trying every combination.

The search range was expanded significantly to $K_p \in [0.1, 4.0]$, $K_i \in [0.0, 0.02]$, and $K_d \in [0.5, 8.0]$, and run for 600 trials. I tuned it using 20 randomly generated profiles (with fixed seeds) so the tuner sees a diverse set of scenarios without ever touching the evaluation scenario.

The score function was also improved to penalize overshoot, meaning getting *closer* than the desired distance, which is a safety risk not captured by mean error alone.

The best PID gains found were:

$$K_p = 4.0, \quad K_i \approx 0, \quad K_d = 8.0$$

Method	Scenario	Mean Error (m)	Min Distance (m)	Mean Jerk (m/s ³)	Effort	Collisions
Naive ($K_p = 2.0, K_i = 0, K_d = 0$)	Default	6.49	8.56	1.87	110.10	0
Grid Search ($K_p = 0.5, K_i = 0.002, K_d = 2.0$)	Default	2.01	16.37	0.35	30.14	0
Bayesian ($K_p = 4.0, K_i \approx 0, K_d = 8.0$)	Default	0.99	19.51	0.43	33.69	0
	Hard	0.93	19.42	0.99	46.30	0

Table 1: Performance of PID controllers across tuning methods and scenarios.

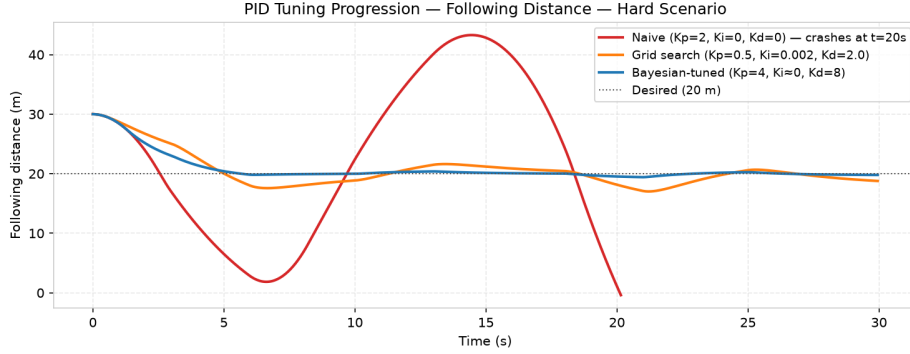


Figure 3: Following distance comparison between the naive PID controller, Grid Search, and Bayesian tuned PID controller.

Observations

Bayesian optimization found significantly better gains than grid search, cutting mean tracking error by more than half and keeping the minimum following distance much closer to the desired 20 m.

A notable finding is that $K_i \approx 0$ was consistently optimal. This makes sense for this problem: the lead car scenario is deterministic and the proportional and derivative terms are sufficient to track it. There is no persistent steady-state error that requires integral correction.

The optimal gains, $K_p = 4.0$ and $K_d = 8.0$, also consistently hit the ceiling of the search range. This is because the acceleration clipping, $[-5, 3] \text{ m/s}^2$, acts as a natural stabilizer. The car physically cannot react fast enough to oscillate, so higher gains continue to improve tracking without causing instability.

However, the controller still reacts *after* the distance error has already changed. During sudden braking by the lead car, the following distance briefly drops before the ego car catches up. This is a fundamental limitation of a pure distance-error PID controller: it cannot anticipate a closing gap, only respond to one that is already happening.

6 Reinforcement Learning Setup

I wrapped the ACC simulator as a Gymnasium environment and implemented standard `reset()` / `step()` interface so that any Stable-Baselines3 algorithm could be plugged normally.

Observation Space

The agent observes distance error, relative speed, and the absolute speeds of each vehicle at each timestep, as shown below:

$$s = [e_d, v_{\text{rel}}, v_{\text{ego}}, v_{\text{lead}}]$$

where $e_d = d - d_{\text{desired}}$ is distance error, $v_{\text{rel}} = v_{\text{lead}} - v_{\text{ego}}$ is relative speed, and v_{ego} and v_{lead} are absolute speeds of each vehicle. This 4d observation allows the agent to detect whether the gap is closing ($v_{\text{rel}} < 0$), whether it is too close ($e_d < 0$), and how fast it is moving in absolute terms.

Action Space

The action is the scalar ego-car acceleration:

$$a \in [-5, 3] \text{ m/s}^2$$

This value is clipped to realistic braking and acceleration limits.

For the three driving styles, I kept the same observation and action spaces. The only difference was the reward weighting. Aggressive driving used a stronger tracking emphasis, safe driving used a stronger close-distance penalty, and smooth driving used stronger jerk and acceleration penalties.

Algorithm: PPO

I used PPO (Proximal Policy Optimization) and implemented it with Stable-Baselines3. PPO was introduced by Schulman et al. [4]. PPO maintains two networks: a *policy network* (actor) that maps observations to actions, and a *value network* (critic) that estimates the expected future return from a given state. At each iteration, PPO collects $n_{\text{steps}} = 2048$ environment steps using the current policy, computes advantages using the value network, then updates both networks with minibatch gradient descent. The “proximal” constraint clips the policy update ratio to $[1 - \varepsilon, 1 + \varepsilon]$ with $\varepsilon = 0.2$, preventing any single update from changing the policy too drastically and destabilizing training.

More specifically, PPO clips the probability ratio inside its training objective. This does not guarantee that the policy can never change too much, but it makes the updates more conservative than a completely unconstrained policy-gradient update [4].

Training Scenario

I had a difficult time deciding the training scenario. For the earlier runs, I trained on the fixed **default** scenario (Run 1) or **hard** scenario. However as expected, it created an overfitting problem where the agent just memorized the timing of braking events instead of learning to react to distance error and relative speed.

To prevent this, I implemented a randomized scenario for all the RL training from Run 2 onward. At the start of each episode, a new lead car acceleration profile is sampled from random phase durations of 2–7s and accelerations drawn from $\{-4, -3, -2, -1, 0, 0, 0, 1, 2\} \text{ m/s}^2$, with zero weighted three times to represent realistic cruising. This way, the agent is forced to generalize across profiles and cannot memorize any fixed sequence.

The two fixed scenarios, **default** and **hard**, were then reserved exclusively for evaluation, and neither were used during RL training.

7 Reward Function Design and Iteration

The biggest challenge I came across for this project was designing the reward function. I went through 4 versions of the reward across five runs, with each failure revealing another specific issue.

Reward v1 (Run 1 + 2): Very large collision penalty

My first reward function was based on an intuitive idea:

$$r = \text{distance reward} - \text{unsafe distance penalty} - \text{jerk penalty} - \text{acceleration penalty} - \text{collision penalty}$$

More specifically:

$$r = -|e_d| - 2|e_d|\mathbf{1}[d < d_{\text{des}}] - 0.05|j| - 0.02a^2 - 1000\mathbf{1}[\text{collision}]$$

where:

- $e_d = d - d_{\text{des}}$ is the distance error.
- d is the current following distance.
- d_{des} is the desired following distance.
- j is jerk, meaning how suddenly acceleration changes.
- a is the ego car’s acceleration.
- $\mathbf{1}[\cdot]$ means the penalty only turns on when the condition is true.

Basically, the reward punishes the agent for being far from the desired distance, being too close to the lead car, driving jerkily, using large acceleration, and crashing.

The -1000 collision penalty was supposed to be a safety signal - crashing is really bad! However, in practice it failed horrendously because it fires exactly once at the moment of impact, then the episode terminates. Every step leading up to the crash (5 m gap or 2 m gap), received the same small tracking penalty as a step at 19 m. PPO’s gradient update traces backward through many steps; attributing a single terminal penalty to the actions that caused a dangerous approach 5–10 steps earlier is too noisy to learn from reliably. The agent received no signal that “close is dangerous” until it was already too late.

Result: both Run 1 and Run 2 crashed consistently, with 0/1 and 0/1 converged.

Reward v3 (Run 3): Proximity Ramp

I replaced the harsh collision penalty with a smooth ramp that turns **before** collision:

$$r = -w_{\text{close}}|e_d| - P_{\text{ramp}}(d) - 0.05|j| - 0.02a^2 - 500\mathbf{1}[\text{collision}]$$

where

$$P_{\text{ramp}}(d) = \begin{cases} 10(15 - d), & \text{if } d < 15 \text{ m,} \\ 50(5 - d) + 10(15 - d), & \text{if } d < 5 \text{ m,} \\ 0, & \text{otherwise.} \end{cases}$$

The tracking penalty was also made asymmetric. If the ego car was too close to the lead car, I used a larger weight, $w_{\text{close}} = 3$. If it was too far away, I used a smaller weight, $w_{\text{close}} = 1$. This made sense because being too close is much more dangerous than being too far.

At a 10 m gap, the proximity ramp gives a penalty of -50 per step. Since the simulation uses $\Delta t = 0.1 \text{ s}$, one second contains 10 steps, so this adds up to about -500 over one second. This is similar in size to the old collision penalty, but now the penalty is spread out over the dangerous approach instead of only happening at the crash. This gave the agent a much clearer signal that it should move away from the lead car before it was too late.

Run 3 produced the first policies that actually converged. In the training logs, `ep_len_mean` reached 300, which means the agent was completing full episodes without crashing. The `explained_variance` also reached 0.946, which suggests that the value network was doing a good job predicting returns, so PPO had a cleaner learning signal.

Run 3 Seed 2: Reward Hacking

For Run 3, I tested 5 different seeds, and seed 2 exposed a major flaw in the reward function. This seed had a very large error, with $\text{MAE} = 96.6 \text{ m}$, and the minimum following distance was still 30 m. Instead of actually following the lead car, the ego car basically slowed down and let the lead car drive away. By the end of the episode, the gap had grown to more than 248 m.

The problem was that my reward punished the car heavily for getting too close, but it did not punish it enough for being way too far away. The proximity ramp only turned on when $d < 15 \text{ m}$, so once the ego car stayed far behind the lead car, it avoided the main safety penalty completely. The normal tracking penalty was not strong enough to stop this. From the agent’s point of view, stopping far behind the lead car was a pretty good strategy: it had zero crash risk, no proximity penalty, and only a moderate tracking cost.

This is a clear example of reward hacking. The agent was technically doing what the reward function encouraged, but it was not doing what I actually wanted. I wanted it to follow the lead car at a safe distance, but the reward accidentally allowed it to avoid the task by staying extremely far away.

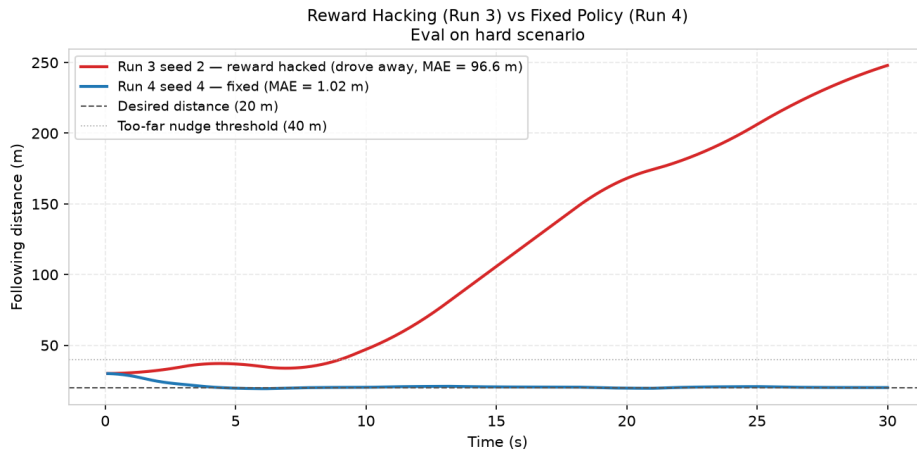


Figure 4: Reward hacking in Run 3 seed 2 versus the fixed policy in Run 4 seed 4. The hacked policy let the ego car stop while the lead car drove away, growing the gap to 248 m. After adding the too-far nudge in Run 4, the agent tracked the desired 20 m following distance throughout the episode.

Reward v4 (Run 4): I tried to fix the reward hacking

I added a small penalty when the ego car gets too far away:

$$P_{\text{far}}(d) = 0.5 \max(0, d - 40)$$

This means nothing happens if the distance is under 40 m. But once the ego car is more than 40 m behind the lead car, it starts losing points. For example, at 60 m, the penalty is:

$$0.5(60 - 40) = 10$$

So the car gets a small penalty for being too far away.

The important part is that I kept this penalty much smaller than the close-distance safety penalty. Being too close is still much worse than being too far. For example, being at 5 m is extremely dangerous, so the reward gives a much larger penalty there. Being at 60 m is bad because the car is not following properly, but it is not unsafe in the same way.

So the goal of Run 4 was not to make the car aggressively chase the lead car. It was just to close the loophole where the agent could completely run away and still avoid punishment.

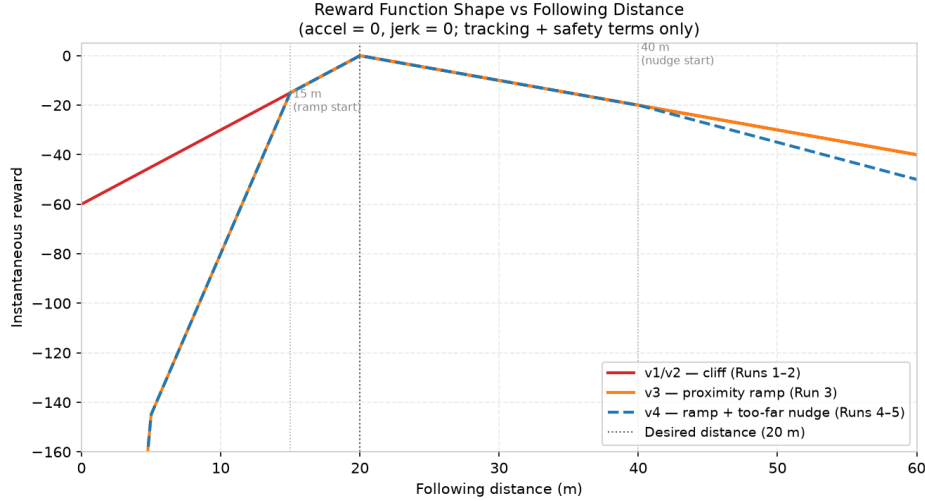


Figure 5: Reward shape across the different reward versions. The proximity ramp makes close distances much more costly, while the final too-far nudge prevents the agent from reward-hacking by staying extremely far away.

Distance (m)	Error (m)	Reward (per step, $a = 0$)	Zone
5	-15	-145.0	Danger
15	-5	-15.0	Warning
20	0	0.0	Target
30	+10	-10.0	Tracking
40	+20	-20.0	Tracking
60	+40	-50.0	Too far

Table 2: Reward values at different following distances. The table shows that getting dangerously close is punished much more than being too far away, but the too-far loophole is still penalized.

Failed VecNormalize Attempts

Since Run 3 only had 2/5 successful seeds, I tried using SB3’s **VecNormalize** to rescale the observations. My thought was that the neural network might be struggling because the inputs had different sizes, like distance error being much larger than speed.

This did not help. When I normalised both observations and rewards, 4/5 seeds crashed because the safety penalties became weaker. When I normalised observations only, the results were still worse than Run 3, and one repeat attempt crashed 5/5 seeds.

In the end, I concluded that **VecNormalize** was solving the wrong problem. The reward penalties were already carefully scaled, especially the close-distance safety penalty. The instability seemed more related to random seed and policy initialization, so I went back to the normal environment.

8 Driving Styles

Once I had a final reward function that could actually learn stable ACC behaviour, I wanted to see whether I could use the same setup to create different driving styles. I made three versions: aggressive, safe, and smooth. The simulator, observation space, and action space stayed the same. The main difference was how I weighted tracking, safety, acceleration, and jerk.

Style	Main priority	Expected behaviour	How I changed it
Aggressive	Tight tracking	Closes the gap faster and reacts more strongly	Higher tracking emphasis and lower relative comfort penalty
Safe	More distance from the lead car	Brakes earlier and avoids close gaps more strongly	Higher close-distance penalty and optionally a larger desired gap
Smooth	Passenger comfort	Changes acceleration more gradually	Higher jerk and acceleration penalties

Table 3: The three driving styles and what each one prioritizes.

The aggressive style cares more about quickly closing the gap and staying near the desired following distance, so it is more willing to use stronger acceleration or braking. The safe style cares more about avoiding close distances, so it reacts more strongly when the ego car gets too close and is more willing to stay farther behind. The smooth style puts more weight on jerk and sudden acceleration changes, so it reacts more gradually even if that causes slightly more tracking error.

I made these styles by changing the weights for tracking error, close-distance safety, jerk, and acceleration. So they are not completely different algorithms. They are just different versions of the same controller objective. This is similar to personalized ACC research, where different cost functions are used to represent different driver preferences [3].

A general version of the style reward can be written as:

$$r_t^{(s)} = -w_e^{(s)}|e_{d,t}| - P_{\text{close}}^{(s)}(d_t) - P_{\text{far}}^{(s)}(d_t) - w_j^{(s)}|j_t| - w_a^{(s)}a_t^2 - C_{\text{collision}}\mathbf{1}[\text{collision}]$$

where s is the selected style. The important part is that the simulator does not change. Only the priorities change.

Style	Desired gap (m)	Jerk weight	Effort weight	Close-dist weight
General (Run 4)	20	0.05	0.02	3.0
Aggressive	12	0.01	0.005	4.0
Safe	28	0.05	0.02	5.0
Smooth	20	0.25	0.10	3.0

Table 4: Key reward weights for each driving style. All styles use the same proximity ramp and too-far nudge from reward v4.

Style Results

Each style was trained with a 5-seed sweep (3M steps, randomized scenario) and evaluated on the hard scenario. The best non-crashing seed was selected per style.

Style	Best seed	MAE (m)	Mean dist (m)	Min dist (m)	Mean jerk (m/s ³)
General (Run 4, baseline)	4	1.024	19.29	19.29	1.179
Aggressive	3	2.968	12.40	1.80	1.246
Safe	3	2.479	24.47	16.82	1.104
Smooth	2	4.897	23.90	14.89	0.874

Table 5: Best-seed results per driving style, evaluated on the hard scenario. MAE is measured relative to each style’s own desired gap.

Style	Crashes (5 seeds)	Converged seeds
General	1/5	3/5
Aggressive	4/5	1/5
Safe	1/5	4/5
Smooth	3/5	2/5

Table 6: Seed convergence across driving styles. Aggressive is the hardest to train because the tighter desired gap puts the agent closer to the collision boundary.

The results match intuition. Aggressive achieved the tightest mean following distance (12.40 m, near its 12 m target) but crashed 4/5 seeds — training close to the collision boundary makes the safety constraint harder to satisfy. Safe had the best crash rate (1/5) and consistently maintained a larger buffer. Smooth had the lowest jerk of any style (0.874 m/s³) at the cost of higher tracking error, which makes sense since it is penalising rapid acceleration changes.

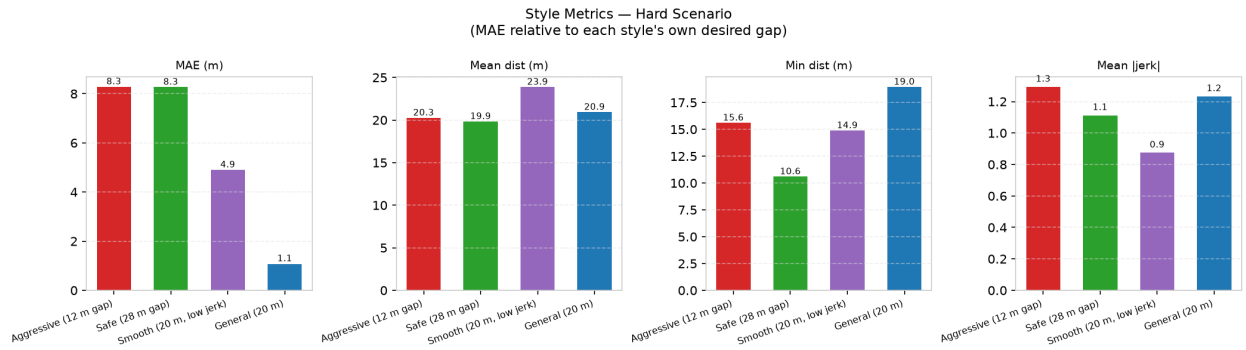


Figure 6: Per-style metric comparison on the hard scenario (best seed per style). MAE is relative to each style's own desired gap. Aggressive has the highest crash rate; smooth has the lowest jerk; safe maintains the largest safety buffer.

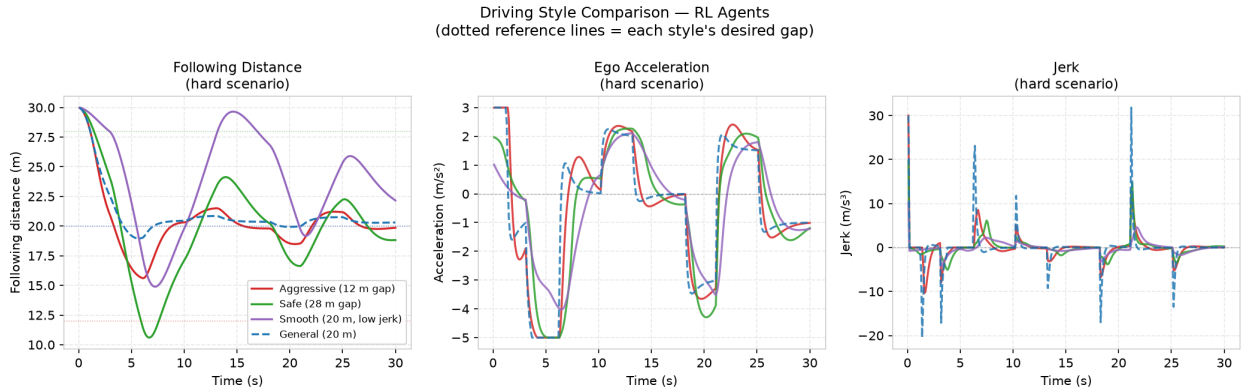


Figure 7: Following distance, acceleration, and jerk for each driving style on the hard scenario. Dotted reference lines show each style's desired gap. Aggressive tracks a tighter gap with sharper inputs; safe stays well back; smooth produces the gentlest acceleration profile.

9 Experimental Results

Training Progression

Table 7 summarises all training runs in order.

Run	Steps	Scenario	Reward	Best MAE	Crashes
1	100k	Default (fixed)	v1 (cliff)	8.07m	1/1
2	500k	Randomised	v1 (cliff)	8.64m	1/1
3	1.5M	Randomised	v3 (ramp)	1.09m	1/5 seeds
4	1.5M	Randomised	v4 (ramp + nudge)	1.02m	1/5 seeds
5	3M	Randomised	v4	1.08m	3/5 seeds

Table 7: Summary of all RL training runs. Runs 3–5 report results from 5-seed sweeps.

Run 1 and Run 2 failed entirely due to the binary collision cliff: the reward provided no gradient signal before impact. The transition to the proximity ramp in Run 3 was the critical change. The first converging policies emerged, and training logs confirmed `ep_len_mean` = 300 and `explained_variance` = 0.946. Run 4 closed the reward-hacking loophole from Run 3 and improved seed convergence from 2/5 to 3/5. Run 5, with 3M steps, did not improve over Run 4. Seeds 0, 2, and 3 crashed at approximately the same episode lengths, around 60–78 steps, in both runs, confirming they were trapped in local minima rather than simply undertrained.

Seed Stability

Because PPO is sensitive to weight initialisation on continuous-control tasks, all runs from Run 3 onward were evaluated as 5-seed sweeps. Deep RL results can change a lot across random seeds, so only reporting one good run can be misleading [5].

I separated the seed outcomes into a few categories:

- **Crashed:** the episode ended before 300 steps because the cars collided.
- **Safe completion:** the policy finished all 300 steps without collision.
- **Loose convergence:** it finished safely, but the tracking error was still large.
- **Strict convergence:** it finished safely and stayed close to the desired gap.

Table 8 shows Run 4 results.

Seed	MAE (m)	Min dist (m)	Jerk (m/s ³)	Steps	Verdict
0	8.978	−0.857	0.668	73/300	Crashed
1	10.153	18.095	0.849	300/300	Loose convergence
2	2.379	20.907	1.022	300/300	Converged
3	1.298	19.826	1.132	300/300	Converged
4	1.024	19.293	1.179	300/300	Best
Mean	4.767	15.453	0.970	—	—
Std	3.962	8.205	0.189	—	—

Table 8: Run 4 seed sweep results, evaluated on the hard scenario.

The variance is high because 2/5 seeds never learned a useful policy.

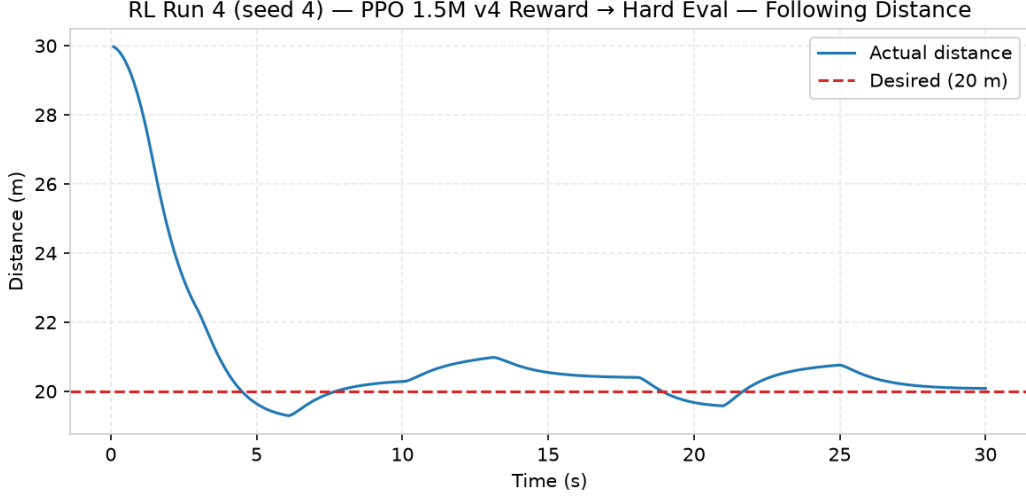


Figure 8: Following distance over time for the best Run 4 seed (seed 4) on the hard scenario. The policy maintains close to the desired 20 m gap throughout, including during the emergency brake at $t = 3\text{--}6$ s and the hard brake at $t = 18\text{--}21$ s.

This is also why the best-seed result and the full seed sweep answer two different questions. The best seed shows whether PPO *can* learn a good controller. The seed sweep shows whether it learns one *reliably*. In my case, PPO could get close to PID, but it did not do that consistently.

Final Comparison: PID vs. RL

Controller	MAE (m)	Min dist (m)	Jerk (m/s^3)	Crashes	Success	vs. PID
PID (Bayesian-tuned)	0.927	19.42	0.989	0	100%	—
RL best seed (Run 4)	1.024	19.29	1.179	0	100%	+10.5% error

Table 9: PID vs. best RL policy evaluated on the hard scenario.

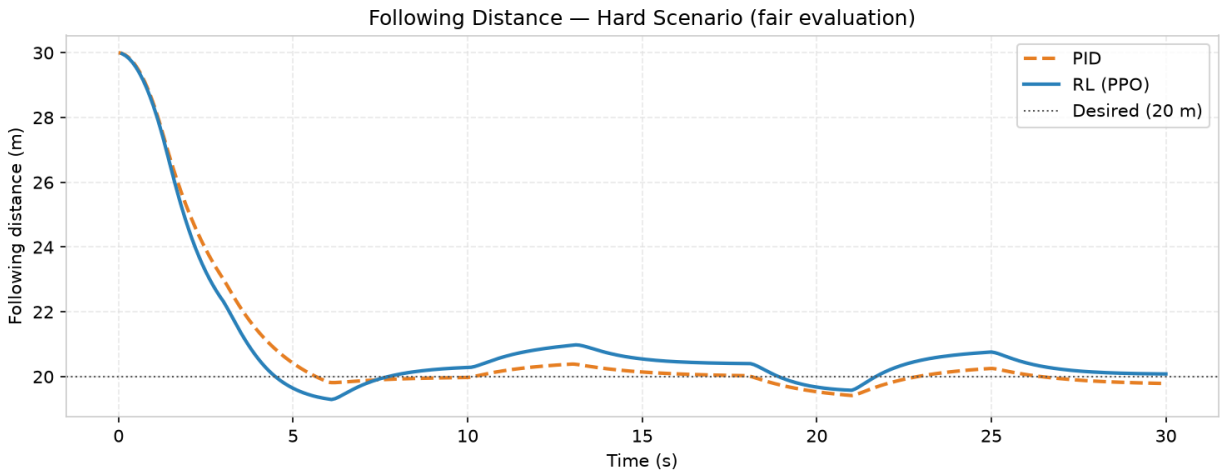


Figure 9: Following distance over time for PID and the best RL policy on the hard scenario. Both controllers avoid collision; the RL policy shows slightly more oscillation during the emergency braking phase.

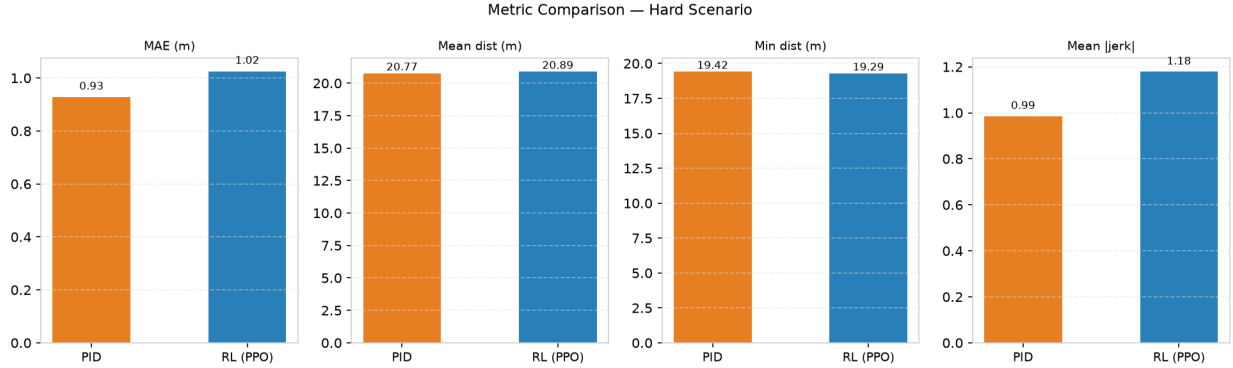


Figure 10: Metric comparison between PID and best RL policy. PID edges out RL on MAE and jerk; the two are nearly identical on minimum following distance.

The final result was actually pretty close. The Bayesian-tuned PID controller achieved a mean absolute error of 0.927m, while the best RL policy achieved 1.024m. This means the RL policy had about 10.5% more tracking error than PID, but it still avoided crashes and stayed close to the desired following distance.

The more interesting part is that the RL policy was trained on randomised lead-car behaviour and then tested on the hard scenario. Evaluated on 20 unseen randomised profiles, the policy achieved a 100% success rate with $MAE = 1.25 \pm 0.35$ m. This suggests that the RL policy was not just memorising one fixed scenario; it learned a general reactive strategy for following the lead car.

Generalization vs. Training Reliability

There are two different things I wanted to separate:

- **Policy generalization:** does one trained policy still work on new lead-car behaviour?
- **Training reliability:** if I train the exact same setup again, do I get another good policy?

My final selected policy generalized pretty well because it had a 100% success rate across 20 unseen randomized profiles. However, the seed sweep showed that the training process itself was not very reliable. Lin et al. also found that DRL-based ACC worked better inside the training range and became weaker outside of it [1]. So even if one policy generalizes, I still cannot assume every training run will produce that policy.

10 Discussion

PID vs. RL

The PID controller was the stronger and more reliable baseline. Once the gains were tuned, it gave the same result every time and was easy to understand. If the ego car was too far, it accelerated; if it was too close, it braked. The behaviour was very direct.

The RL policy was messier, but still got surprisingly close. The best RL seed reached 1.024m MAE on the hard scenario, compared to 0.927m for PID. So PID was still better, but RL was not completely far off.

Reward Engineering

Reward design was the hardest part of the RL side. My first reward used a big crash penalty, but that only punished the agent after it had already crashed. It did not really teach the agent that getting too close was already dangerous.

The proximity ramp was the key fix. Instead of only punishing collision, it punished the car more and more as it got closer to the lead car. This gave the agent a much clearer safety signal before crashing.

Run 3 also showed reward hacking. One seed learned that it could avoid large penalties by slowing down and letting the lead car drive far away instead of actually following it. Technically, it avoided the close-distance penalty, but it completely missed the point of adaptive cruise control. Adding a small too-far penalty helped close that loophole.

Driving Styles

The three driving styles made the tradeoffs easier to see. The aggressive style tries to recover the desired gap faster, but this can increase acceleration, jerk, or how close the ego car gets. The safe style is more conservative around the lead car, but it may stay farther away than necessary. The smooth style reduces sudden acceleration changes, but it can react more slowly and have slightly higher tracking error.

This also showed why there is not really one controller that is the best at everything. It depends on whether I care most about tracking accuracy, safety margin, or passenger comfort. That is why I kept all of the evaluation metrics instead of only looking at MAE.

Limitations and Tradeoffs

The simulator is very simplified. The cars only move in one dimension, and there are no lane changes, sensor noise, reaction delays, road conditions, or multiple vehicles. So this should be treated as a simplified control experiment, not a realistic self-driving system.

The biggest RL limitation was seed instability. Some seeds learned good policies, while others crashed or learned bad behaviour. This matched what I saw in the adaptive cruise control RL papers too: deep RL can perform well, but reliability and generalisation are still major issues. Lin et al. also found that their RL controller worked better within the training range and became weaker outside of it [1]. In my project, the seed sweep made this really obvious because the same algorithm and reward could still lead to different final behaviours depending on the random seed.

There was also a tradeoff between tracking, safety, and comfort. A controller can get low distance error by following closely, but that can become unsafe. A cautious controller may be safer, but it might stay too far away. That is why I used multiple metrics like MAE, minimum distance, jerk, effort, and collisions instead of only looking at one number.

Another limitation is the algorithm choice. I used PPO because it is stable, common, and easy to use in Stable-Baselines3, which made sense for a first working version. However, adaptive cruise control is a continuous-control problem because the action is acceleration. The papers I looked at often used continuous-control methods like DDPG for this type of task [1, 6], so a better extension would be to compare PPO with DDPG, TD3, or SAC. These might produce smoother acceleration and more reliable training.

11 Future Work

If I kept working on this, the main things I would try next are:

- use the same simulation time step for PID and RL;
- compare PPO with DDPG, TD3, and SAC using the same training budget;
- train more than five seeds and report confidence intervals;
- add sensor noise, action delay, and actuator lag;
- train one policy that takes the driving style as an input;
- learn the style preferences from demonstrations instead of choosing the weights manually;
- use a speed-dependent desired gap instead of always using exactly 20 m; and
- add an emergency-braking safety override.

A speed-dependent following distance would probably be more realistic. For example:

$$d_{\text{desired}} = d_0 + \tau v_{\text{ego}}$$

where d_0 is a minimum gap and τ is the preferred time headway. The safe style could use a larger τ , while the aggressive style could use a smaller one.

12 Conclusion

This project compared a Bayesian-tuned PID controller with a PPO reinforcement learning policy for a simplified adaptive cruise control task. PID was more stable, interpretable, and reliable, achieving 0.927 m MAE on the hard scenario. The best RL policy was slightly worse at 1.024 m MAE, but it still avoided crashes and generalized to unseen randomized scenarios.

The main lesson was that RL only works if the reward actually matches the behaviour I want. A simple crash penalty was not enough because it punished the agent too late. The continuous proximity ramp was the change that actually made learning work. The reward-hacking failure also showed how easily an agent can exploit a loophole if the reward is missing something.

The aggressive, safe, and smooth modes also showed that there is not only one correct ACC behaviour. The aggressive version prioritizes tighter tracking, the safe version prioritizes more distance from the lead car, and the smooth version prioritizes lower jerk. Each one improves one thing by giving up a little somewhere else.

References

- [1] Y. Lin, J. McPhee, and N. L. Azad, “Comparison of Deep Reinforcement Learning and Model Predictive Control for Adaptive Cruise Control,” *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 2, pp. 221–231, 2021. doi: 10.1109/TIV.2020.3012947.
- [2] L. C. Das and M. Won, “SAINT-ACC: Safety-Aware Intelligent Adaptive Cruise Control for Autonomous Vehicles Using Deep Reinforcement Learning,” in *Proceedings of the 38th International Conference on Machine Learning*, vol. 139, pp. 2445–2455, 2021.
- [3] M. F. Ozkan and Y. Ma, “Personalized Adaptive Cruise Control and Impacts on Mixed Traffic,” in *Proceedings of the American Control Conference*, pp. 412–417, 2021.
- [4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [5] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep Reinforcement Learning That Matters,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [6] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous Control with Deep Reinforcement Learning,” *arXiv preprint arXiv:1509.02971*, 2015.